2章
データドメインを利用したデータの整理

1章で述べたトレンドの影響で、データ管理のあり方を見なおす必要があります。前章では、アプリケーションデータをコピーする際に生じる密結合や、生データを対象とした分析を行うことの難しさについて述べてきました。また、統一化の問題や統合データウェアハウス構築に伴う多大なコスト、アジリティへの影響についても見てきました。1章で述べたように、ドメインや、チーム、ユーザー自身が簡単かつ安全にデータを流通、消費、利用できるようにする手法にシフトする必要があります。データをビジネスに近づける戦略と組織改革が必要です。この作業をサポートするプラットフォームや、プロセス、パターン、標準インタフェースが必要です。大規模に対応できるデータ管理アーキテクチャが必要です。この章では、データドメインを用いてどのようにランドスケープを整理するかというところから議論を深めていきます。

しかしながら、その話題に入る前に、アプリケーションがどのように設計され、どのように連携するかについて、 一般的に知られている原則をいくつか掘り下げてみます。その後、アプリケーションの内部アーキテクチャに焦点を当て、そこから何を学べるかを考えます。その次に、ドメインに目を向け、ドメイン駆動設計とビジネスアーキテクチャについて見ていきます。ドメインのさまざまな特徴、データ管理に関する主要な原則、ドメインオーナーシップの責務について議論します。この章が終わる頃には、データドメインとは何か、なぜ重要なのかを理解できているはずです。この背景となる理論は、新しいアーキテクチャの中核となる要素を理解するのに、必要不可欠なものです。この後の章では、パターン、設計、ダイアグラム、ワークフローをより詳しく見ていき、このドメイン指向アーキテクチャがデータ管理のすべての領域をどのように結び付けているのかを徐々に説明していきます。

2.1　アプリケーション設計の出発点

深く掘り下げる準備として、ここでは広く知られており、本書を通じて何度も参照することになる、アプリケーション設計の重要な原則をいくつか紹介します。

2.1.1　アプリケーションはそれぞれデータストアを持つ

データ管理やアプリケーション間のデータ移動という観点で考えれば、アプリケーションごとに何らかの形でデータを保存していると考えることができるでしょう。そのため、データを使用する場合は、必ずアプリケーションのデータベースにデータを保存する必要があります。確かに、データを別の場所に保存するステートレスなアプリケーションや、同じデータベースを共有するアプリケーション、インメモリ処理を行うアプリケーションなどがあるかもしれません。しかし、それでもすべてのアプリケーションは、どこかにデータを保存する必要があるということです。

2.1.2　アプリケーションは常に一意である

ビジネスの課題に同じものは決してありません。1章では、アプリケーションは**特有の**課題を解決するために使用されると述べました。それぞれのアプリケーションのデータは一意であり、ほかのアプリケーションのデータやコンテキストとは異なります。これはアプリケーションの開発方法が関係しています。アプリケーションの設計・開発にはいくつかの段階があります[†1]。まず、ビジネス要件とニーズについて概念的に考えることから始まります。次に、ビジネスに関する知見を論理的なアプリケーションデータモデルに変換します。このデータモデルは、ビジネス要件をより形式的で具体的に表現したものです。最後に、アプリケーションの実際の設計となる物理的なアプリケーションデータモデルを作成します。このモデルは、論理設計の非機能要件を基にします。

[†1] データ統合とモデル化の基本を学びたいのであれば、このトピックに関する筆者のブログ記事（https://oreil.ly/PUT4t）をご一読ください。

すべてのコンテキストは一意です。それはビジネス上の課題を解決するためのアプリケーションやデータについても同じことが言えます。それぞれのビジネス上の課題には、それぞれ固有の知見が必要とされます。これはドメインという概念につながりますが、その前に、データの出所は、データ管理にとってどういう意味を持つのかを考えてみましょう。

2.1.3　ゴールデンソース

断片化され分散化された環境では、オリジナルで一意なデータについて、権威あるデータソースがどこにあるのかを調べることが難しい場合があります。そのため、データがどこから来たのか、どこで管理されているのかを知ることが重要になります。本書を通じて、**ゴールデンソース**という概念を使用します[†2]。ゴールデンソースは、すべての信頼できるデータを、ある特定のコンテキストの中で管理している、権威ある**アプリケーション**のことです[†3]。

[†2] SoR（System of Record）という言葉を使う人もいますが、これはあるデータ要素や情報についての権威ある情報源を意味します。

[†3] 権威あるアプリケーションを特定するのは簡単ではありません。権威あるソースの中には、複雑なパターンの中に隠れていて、この複雑なパターンを理解しないと見つけられないものもあります。ゴールデンソースについては、Andy Graham氏著“Mastering Your Data”（Koios, 2015）を参照してください。

一意なデータのみを、しかも、常にゴールデンソースから直接取得することで、データに対する完全で一貫性のある説明責任を担うことができます。これはデータガバナンスを強化するために不可欠です。さまざまなドメインが物理的なデータセットやデータ設計のオーナーシップを主張することがあるためです。したがって、ゴールデンソースを特定することが出発点となります。ゴールデンソースはデータ品質にとっても重要です。ゴールデンソースは、データが作成されたり、システムに最初に入力されたりする場所です。ゴールデンソースでデータ品質の問題を解決すれば、データは常にこのソースから流通するため、ほかの場所で同じ品質問題が発生することはありません。

2.1.4　データ統合のジレンマ

アプリケーションには一意のコンテキストがあるため、あるアプリケーションから別のアプリケーションにデータを移動する際には、常に変換が必要となります。つまり、何らかのデータ統合が常に必要になるということです。ETL（extract, transform, and load、抽出・変換・読み込み）であろうと、ELT

（extract, load, and transform、抽出・読み込み・変換）であろうと、仮想または物理であろうと、バッチまたはリアルタイムであろうと、データ統合のジレンマから逃れることはできません。アプリケーション間でデータを移動する場合、設計やアーキテクチャの種類に関係なく、データ変換は常に必要です。

2.1.5　アプリケーションの役割

アプリケーションは、データプロバイダーまたはデータコンシューマのいずれかになります。また、これから説明するように、両方の役割を果たす場合もあります。あるアプリケーションがデータを使用するためには、別のアプリケーションがデータを作成し提供しなければなりません。これは当たり前のことのように思えるかもしれませんが、多数のアプリケーションやシステム間で、データを統合し流通するという状況では、説明責任を果たす上で重要なことなので良く理解しておいてください。

データコミュニティでは、**ソースシステム整合ドメイン**（source system-aligned domain）や**コンシューマ整合ドメイン**（consumer-aligned domain）といった用語を目にすることがあります。これらの用語は、アプリケーション視点ではなく、ドメイン視点のレベルで使われていますが、提供する側と消費する側という同じ役割のことを指しています。

システムやアプリケーションが、データプロバイダーとして機能するか、データコンシューマとして機能するかによって、話が変わります。図2-1に示すように、異なるアーキテクチャ原則が適用されます。

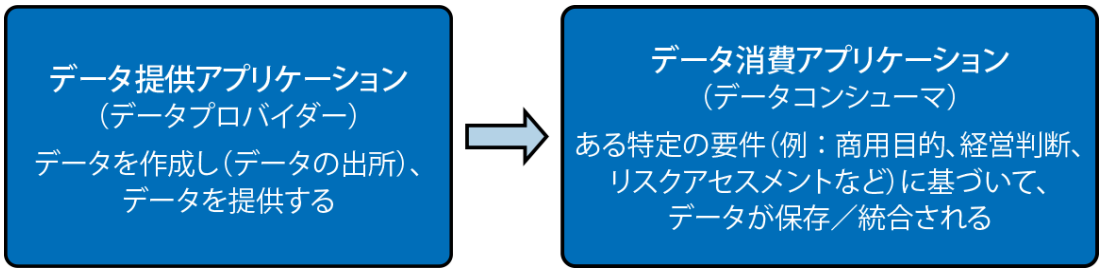

図2-1　データプロバイダーとデータコンシューマというアプリケーションの役割がアーキテクチャを形作る

データプロバイダーとデータコンシューマの役割は、**外部パーティ**にも同じようにあてはまります。外部パーティ、例えばサードパーティは、企業のエコシステムの論理的境界外で運用されるもののことです。一般的には、企業の管理下ではない、別の環境に存在しています。1章では、企業にとってパブリックWebは、再利用可能なサービスを作成・共有することで、収益化できる（オープン）データの宝庫と見なされていることを紹介しました。この新しいコラボレーションの方法は、組織の境界を変えてしまいました。本書で紹介する新しいアーキテクチャでは、外部パーティがデータプロバイダーにもデータコンシューマにもなれるような柔軟性が求められます。外部パーティは常に信頼できるとは限りません。また、既知でもなく、直接関係のあるものばかりでもありません。このため、一般的には、セキュリティ対策を追加する必要があります。

外部パーティからのデータを直接利用してもいいのでしょうか。それとも分離するために新たな抽象化レイヤーを作るべきでしょうか。これは、外部パーティが読者の原則を遵守し、それに準拠することによってデータ提供方法を調整できるのか、あるいは調整する意思があるのかによって変わります。また、外部パーティが設定した標準に準拠しなければならないのかによっても変わります。外部パーティによる調整が保証されない場合は、レイヤーを追加して、抽象化を行い分離しなければなりません。調整が可能であれば、直接消費してもかまいません。

アプリケーションが動作する固有のコンテキスト、データプロバイダーとデータコンシューマの役割、アプリケーション間で常に行われる変換のほかに、アーキテクチャのために何を検討しなければならないのでしょうか。全体として優れたアーキテクチャとはどういうものでしょうか。次の節では、いくつかのソフトウェアアーキテクチャパターンに焦点を当て、そこからどのようなヒントが得られるかを見ていきます。

2.2　ソフトウェアアーキテクチャからの発想

アプリケーション間のデータ統合とは、複雑に絡み合うシステム間において、データ通信と相互運用性を管理することです。エンタープライズアーキテクチャでは、大局的な観点から検討するのが一般的です。しかし、アプリケーション内部のアプリケーションコンポーネントは、どのように連携してデータを流通したり共有したりするのでしょうか。また、そこから何を学ぶことができるのでしょうか。**データメッシュ**（https://oreil.ly/4R2PM）という言葉を作ったZhamak Dehghani氏は、マイクロサービスから着想を得ています[†4]。そこで、このアーキテクチャとその影響を見てみましょう。

[†4] Dehghani氏は、Nextdata設立以前は、Thoughtworksの北米新興技術担当ディレクターを務めていました。Thoughtworksのコンサルタントは、マ

イクロサービスとデータメッシュアーキテクチャを推進する第一人者です。

図2-2に示すように、マイクロサービスは、アプリケーションを小さな独立したサービスで構成し軽量な形式で通信を行うような、ソフトウェア開発のアーキテクチャ手法のことです。つまり、アプリケーションを1つの大きなモノリスとして構築してパッケージ化するのではなく、**マイクロサービス**と呼ばれる小さなコンポーネントに分割するのです。

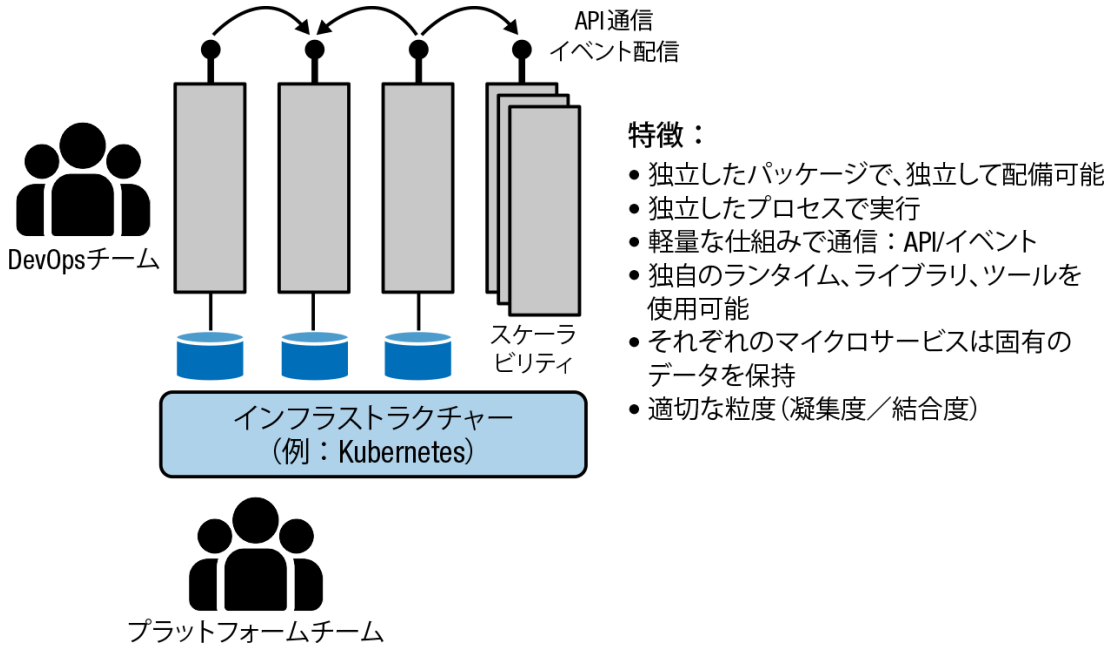

図2-2　マイクロサービスアーキテクチャの概要

マイクロサービスアーキテクチャを使用することの利点は、マイクロサービスを個別に独立して開発、テスト、配備できることです。それぞれのマイクロサービスはそれ専用のランタイムとデータを処理し、専用のプロセスで実行されるため、設計がよりスケーラブルになります。 アプリケーション全体を大規模化するのではなく、個々のアプリケーションコンポーネントを必要に応じてそれぞれ個別に大規模化することができます。プログラミング言語についても同じような柔軟性があります。さまざまな言語を組み合わせて利用でき、コンポーネントごとに最適な言語を選択することができます。例えば、1つ目のマイクロサービスはC#で記述し、2つ目のマイクロサービスはNodeJS、3つ目はPythonで記述する、という風に複数の言語を組み合わせて使うことができます。

すばらしいことだらけのように思えますが、注意すべき点があります。それは、マイクロサービスを増やせば増やすほど、マイクロサービスアーキテクチャの管理は難しくなる、ということです。図2-3に示すように、あるアプリケ

ーションが大規模化し3つの開発チームで保守しなければならなくなり、すべてのマイクロサービスが互いに直接通信している状況を想像してみてください。

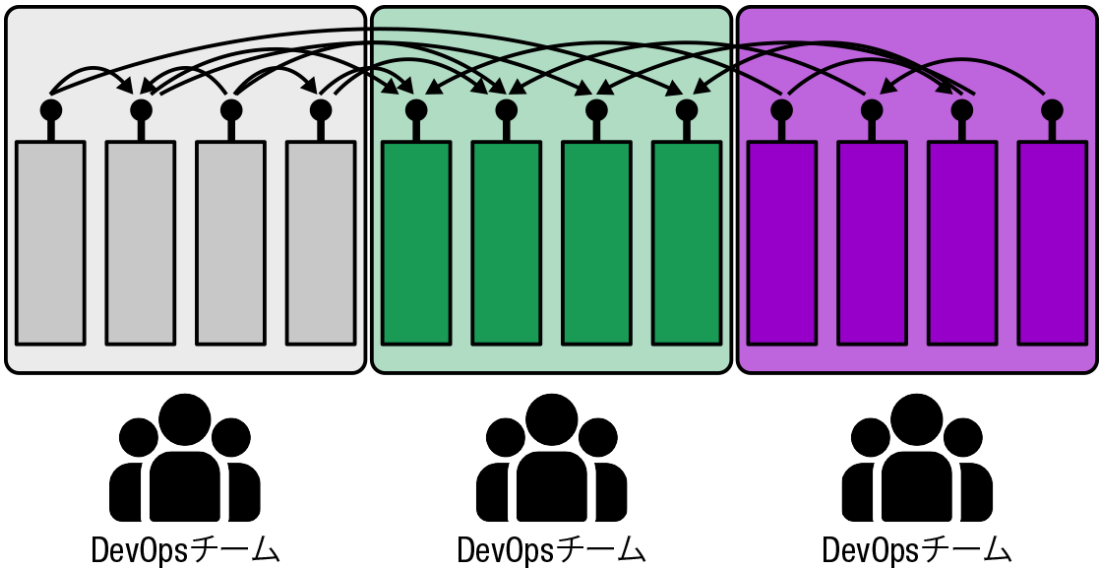

図2-3　複雑な通信

このアーキテクチャでは、マイクロサービスの通信方法のために、アプリケーションが壊れやすくなっています[†5]。もし、あるサービスが故障したり誤動作を始めたりすると、システム全体に波及する可能性があります。そのような事態を避けるために、**サービスメッシュ**と呼ばれる、マイクロサービス間通信を制御するパターンを使うことができます。サービスメッシュは、サービス間の通信を制御するロジックを個々のサービスから独立させ、独自のレイヤーとして抽象化します。サービスメッシュは、監視、ディスカバリー、穏やかな停止（graceful shutdown）、ポリシー施行などの機能を提供します。

[†5] このようなアーキテクチャを表す言葉として、「分散型の大きな泥だんご」という言葉が使われることもあります。多くのマイクロサービスが互いに通信するように構成された結果、複雑な状態機械として機能することになり、その規模が大きくなるにつれて、ますます予測できない状況に陥ってしまいます。

サービスメッシュは、マイクロサービスのグループ化と分離のために使うことができます。このアーキテクチャパターンでは、例えば名前空間（name-space、https://oreil.ly/V40pP）を使って、機能が共有されているかどうかに

基づいてマイクロサービスをグループ化します。グループ（安全な境界）内のマイクロサービスは直接通信できますが、ほかのグループのマイクロサービスと通信するには、サービスメッシュを使用しなければなりません。このパターンでは、サービスメッシュは安全なプロキシとして機能します。図2-4は、ドメイン駆動設計の原則を適用し、サービスメッシュを追加した場合、アーキテクチャがどのように変化するかを示したものです。

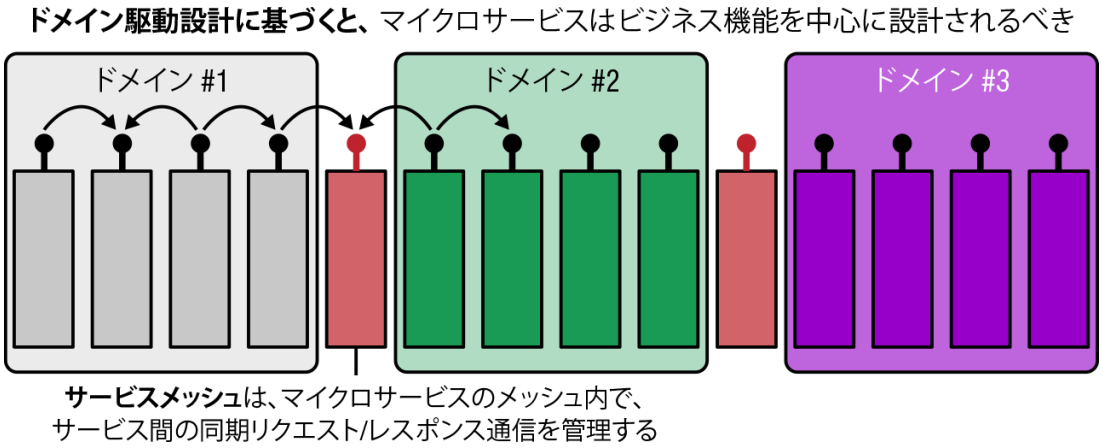

図2-4　サービス間の同期リクエスト/レスポンス通信を管理するためにサービスメッシュを追加

今回変更されたアーキテクチャ設計では、マイクロサービスは凝集度、つまり個々のマイクロサービスがどの程度まとまっているか、どの程度連携しているかを基準にグループ化されています。この構成では、それぞれのDevOpsチームがアーキテクチャのある特定の部分を所有し、コントロールします。このオーナーシップと機能を整合させることで、依存関係がより適切に管理されるため、アプリケーションが不具合を起こすリスクが低減します。また、DevOpsチームが、自分たちのマイクロサービスを、ほかのDevOpsチームのマイクロサービスと通信させたいときは、サービスメッシュを使用してアプリケーションの一部を分離するという方法もベストプラクティスです。

マイクロサービスアーキテクチャにおいて、サービスメッシュは、同期リクエスト/レスポンス通信のセキュリティやロジックなどを管理してくれます。非同期通信の場合は、イベントブローカーに置き換えてもよいでしょう。例えば、マイクロサービス間で通知やイベントを配信する場合などです。利用する通信形態に応じて、どちらか、または、両方を同時に利用できます。

このマイクロサービスの設計で欠けているのは、分析用のデータをサポートする分析データアーキテクチャです。サービスメッシュとデータメッシュを比較すると、サービスメッシュでは、サービスリクエストはプロキシを通してマイ

クロサービス間でルーティングされます。サービスメッシュはサービス間の通信を管理し、それぞれのサービスから分離して別ののレイヤーで抽象化します。一方、データメッシュでは、分析データは**データプロダクト**を通じて提供されます。データプロダクトはデータを変換し、完全性を維持し、ポリシーを管理し、データを提供します。

図2-5からわかるように、データプロダクトには、アーキテクチャ面でサービスメッシュと同じような利点があります。Dehghani氏は、データプロダクトを「新しいアーキテクチャの量子、すなわち、その機能を果たすためのすべてのコンポーネントを持つ、アーキテクチャの最小単位」と表現しています（https://oreil.ly/DdUBD）。

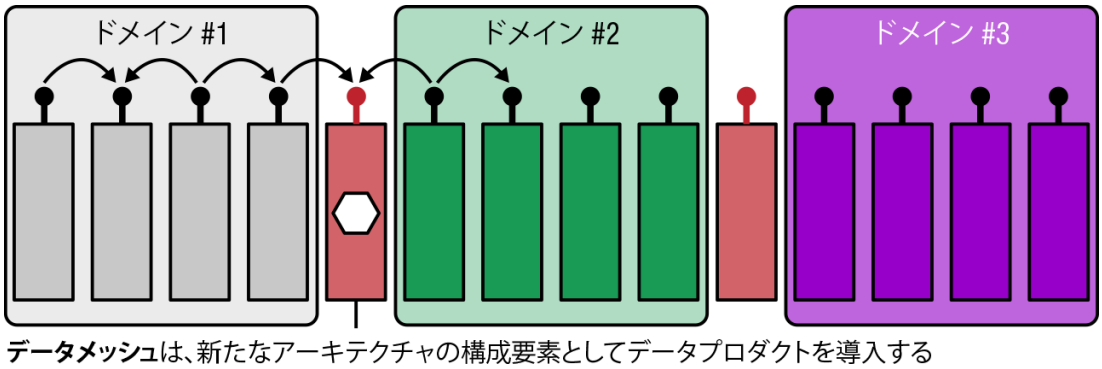

図2-5　データプロダクトを含むデータメッシュアーキテクチャ：個別に配置可能な最小単位

データプロダクトをアーキテクチャの最小単位にするという考え方は非常に良いのですが、すべてのアーキテクチャがマイクロサービスだけで構成されているわけではありません。今日の企業におけるアーキテクチャは、大規模なレガシーシステムをはじめとして、市販の製品（COTS、commercial off-the-shelf）、SaaS、運用データストア、マスターデータ管理ストア、データウェアハウスなど、多くのアプリケーションで構成されているのが一般的です。これらのアプリケーションは、ユーザーや組織単位で共有されることが少なくありません。このような大規模なデータアーキテクチャにおいて、どのように境界を適切に設定すればよいのでしょうか。その答えを、俯瞰して考えてみましょう。

2.3　データドメイン

オーナーシップ境界を設定することは、アーキテクチャを設計する上で非常に重要です。境界をどのように設定するかについては、多くのトレードオフが存

在します。ベストプラクティスとしてよく用いられるのは、**ドメイン駆動設計**（**DDD**、domain-driven design）と呼ばれる方法論を用いることです。次の項では、DDDの基本的な理論について（詳細は省きますが）説明します。その後、ドメインの定義を検討しなおすために、ビジネスアーキテクチャに話を移します。

2.3.1　ドメイン駆動設計

DDDとは、大規模組織向けの複雑なシステムを対象としたソフトウェア開発の手法です。元々はEric Evans氏が著書『エリック・エヴァンスのドメイン駆動設計』（翔泳社、原書“Domain-Driven Design”、Addison Wesley）でまとめたものです。DDDが人気なのは、その高度な実践の数々が、マイクロサービスなどの最新のソフトウェアやアプリケーション開発手法に影響を与えたからです。それでは、いくつか重要な概念を見ていきましょう。

2.3.1.1　コンテキスト境界

コンテキスト境界は、ドメイン駆動設計の中心となる概念です。コンテキスト境界は、ドメインの解決空間に論理的な境界として設定し、複雑性を管理するために使われます。これらの境界は明示的なものになるのが一般的であり、高い凝集度を持つ領域に適用されます。ドメインの依存関係は、アプリケーション、プロセス、関連するデータベース設計、プラットフォームなどのさまざまなレベルに存在します。それぞれのチームにおいて、データなど自分たちだけで変更できる部分と、ほかのチームと調整する必要がある部分とを区別することが重要です。前節でのマイクロサービスの例で見たように、境界を設定することは、チームと開発者が依存関係を効率的に管理するのに役立ちます。

コンテキスト境界は**多態的**です。つまり、その大きさや形は、自分の視点や周囲の状況によって変化します。つまり、コンテキスト境界は、どの視点から見るかによって、位置を変えたり、ほかのコンテキスト境界と重なったりします。つまり、コンテキスト境界を使うときは、どの視点から見ているのかを明確に示す必要があります。そうしないと、ほかの人はどういう意図なのか正確に理解できなくなることでしょう。

ドメインとコンテキスト境界

DDDでは、コンテキスト境界、ドメイン、サブドメインという言葉を区別して使っています。**ドメイン**とは、私たちが取り組もうとしている問題空間のこと

です。ドメインとは、知識、行動、法律、作業がまとめられる場所です。意味的に結合していたり、コンポーネントやサービス間の依存性が存在したりする部分でもあります。多くの場合、複雑さを管理するために、ドメインはサブドメインに分割されます。よくある例は、ドメインを分割して、それぞれのサブドメインを組織のさまざまな部署に対応させるケースです。

すべてのサブドメインが同じというわけではありません。サブドメインは、コアサブドメイン、汎用サブドメイン、サポートサブドメインなどに分類することができます。**コアサブドメイン**は、最も重要なものです。コアサブドメインは、ビジネスを唯一無二のものにする「秘伝のソース」となる材料です。**汎用サブドメイン**は、一般的には既製品で簡単に解決できるような、ビジネス上の課題を表します。**サポートサブドメイン**は、競争上の優位性はありませんが、組織を維持するために必要なサブドメインです。通常、それほど複雑ではありません。

コンテキスト境界は、（コンテキストの）論理的な境界線です。これらは、解決空間に焦点を当てたものです。システムやアプリケーションの設計に関わるものです。コードやデータベースの設計などのことです。ドメインとコンテキスト境界を合わせてもかまいませんが、必須ではありません。コンテキスト境界は、元々技術的なものなので、複数のドメインやサブドメインにまたがることもあります。

論理的な境界線を設定する、つまり、コンテキスト境界を使うのは、責任の所在を明確にして管理しやすくするという考え方を踏まえたものです。同じメンバーが同じ目的に取り組むことで、チームメンバー間のコミュニケーションは効率的になります。ソフトウェアに論理的な境界線を設定することは、ある意味マイクロサービスの考え方と似ています。

企業レベルでは、凝集度が高く、関心事を共有しているアプリケーション（つまり、同じようなタスクを実行したり、同じような責任を持ったりするアプリケーション）を論理的にグループ化します。これらを機能ドメインや、論理グループ、クラスター、組織ケイパビリティと呼ぶ場合もあります。

エンティティやプロセス、アプリケーションをグループ化することは新しい考え方ではありません。「伝統的な」グループ分けとドメイン駆動設計モデルの大きな違いは、DDDでは**強制的に厳しい境界**が設定されることです。コンテキスト境界は独自に進化してもかまいませんが、分離されておかなければならな

いと、Evans氏は主張します。このような分離は、通常、アプリケーションの複雑な内部機能を隠蔽したり（抽象化したり）、安定したインタフェースやレイヤーを提供したりすることで実現できます。

2.3.1.2　ユビキタス言語

DDDにおける**ユビキタス言語**とは、開発者とユーザーによって作り上げられ共有される共通言語です。これは、特定の分野の人々が、自分たちが使用する定義、語彙、専門用語、用語集などを共有することに似ています。ユビキタス言語は、大規模なチーム内や、チーム間の人々をつなげる助けとなります。大規模になると、チーム内やチーム間で、共通の語彙を用いるのが難しくなるからです。ユビキタス言語によって、チーム間でのコミュニケーションの負荷が増えたり、チーム間で用語に一貫性がなかったりするという問題を避けることができます。ユビキタス言語は、ドメイン内でアプリケーションやソフトウェアの設計をサポートするために使われます。そのため、企業全体で唯一の統一言語が存在するとは限りませんが、複数のユビキタス言語に重なる部分が生じることはあります。

> Eben Hewitt氏の“Semantic Software Design”（O'Reilly, 2019）を読まれると良いと思います。この本はデザイン思考を用いて、対になる意味論とアーキテクチャの設計を著しています。

コンテキスト境界とユビキタス言語は深く関連しています。DDDでは、それぞれのコンテキスト境界が独自のユビキタス言語を持つことが想定されているためです。あるコンテキスト境界内で管理されているアプリケーションやアプリケーションコンポーネントは、同じユビキタス言語を使うべきです。コンテキスト境界が大きくなり、チーム全体が同じユビキタス言葉を理解するのが難しくなった場合は、コンテキスト境界をより小さくなるように分割してください。コンテキスト境界が変われば、ユビキタス言語もそれに応じて異なるものになります。経験則では、1つのコンテキスト境界は1つの（アジャイルや

DevOps）チームで管理されます。同じチームのメンバーであれば、現在の状況や、その依存関係のすべてを簡単に認識できるからです。

コンテキスト境界の概念について語るとき、筆者はよく「カルチャー」を使って説明します。異なるチーム間では、言葉の定義や用語は同じではないことは明らかです。コンテキストは特定のものであり、自分たちが知っていることに基づいてるのが一般的です。コンテキスト境界は、それぞれが異なる目的、異なる背景を持ちます。そして、特に重要なことに、異なるカルチャーを持っています。このカルチャーによって、どのように考え、設計し、モデル化するかが大きく変わります。カルチャーの中にはサブカルチャーもあります。サブカルチャーとは、より広いカルチャーの中において、その中でも、目的や慣習を共有するような人々のグループのことです。DDDでも同じことが言えます。コンテキスト境界は、複数のサブドメインや、複数の小さなコンテキスト境界から構築されていてもかまいません。1章で見たように、（ビジネス）コンテキストとアプリケーション設計の間には強い相関関係があります。したがって、コンテキストを表現するために用いられるユビキタス言語は、アプリケーションの設計にも影響を与えます。

DDD手法では、高い凝集度を持つ領域の境界を設定するために、コンテキスト境界を使います。DDDを企業レベルのアプリケーション環境に適用する場合、ドメインの境界はアプリケーションだけでなく、言語、知識、チームリソース、テクノロジーも1つにまとめます。しかし、DDDの本来のユースケースは、ソフトウェア開発（例えばマイクロサービスなど）の観点から複雑なシステムをモデル化することです。このため、データ管理にDDDを用いるのは簡単ではありません。DDDはエンタープライズデータのモデル化に使われることを想定しておらず、データ管理の現場の担当者は、DDDが抽象的で非現実的だと感じてしまいます。メタファーを使って説明しましょう。ドメインとは、読者が住んでいる町のような、共通の関心のある領域だと考えてください。町はドメインに相当し、それぞれの家はサブドメインに相当します。それぞれの家は自分の敷地内に建っていますが、庭や塀に囲まれているため、敷地全体を埋め尽くしているわけではありません。この塀が、コンテキスト境界の境界です。この例では、住宅所有者それぞれが自分の家を維持する責任を負い、その境界内では自由に好きなことができます。しかし、境界（塀）の向こう側では、一定のルールに従わなければなりません。例えば、敷地内への出入りは必ずドライブウェイ（門から玄関までの私道）を通り、車は車道を走り、歩くときは歩道を歩かなければならない、などです。

例えば、データを民主化するためのコンセプトとしてデータメッシュを採用し、より柔軟性を高めるためにドメイン指向のデータオーナーシップの原則を実践するとしたら、これは実際にはどのように機能するのでしょうか。エンタープライズデータモデリングからDDDモデリングへの移行はどのようなものになるでしょうか。データ管理のためにDDDからどのような知見が得られるのでしょうか。

2.3.2　ビジネスアーキテクチャ

筆者としては、大規模なアーキテクチャを設計する際には、まずスコープを精査し、取り組もうとしているビジネス戦略上の問題領域を理解することから始めた方が良いと考えています。技術的な実装方法の細部に取りかかる前に、まずこの作業を行うことが重要です。DDDが提唱しているように、問題領域の間に論理的な境界線を設定することで、責任の所在が明確になり、管理しやすくなるからです。

問題領域をどのようにグループ化するかを決めるには、ビジネスアーキテクチャに目を向けてください。The Business Architecture Guildは、その“Business Architecture Body of Knowledge （BIZBOK Guide）”（https://oreil.ly/imfcf）の中で、ビジネスアーキテクチャを「ケイパビリティ（機能）、エンドツーエンドの価値提供、情報、組織構造、およびこれらのビジネスビューと戦略、製品、ポリシー、イニシアチブ、ステークホルダーとの関係について、全体的かつ多次元的なビジネスビューを表すもの」と説明しています。アーキテクトは、ビジネスアーキテクチャを、原則、ガイダンス、望ましい結果、企業とそのビジネスエコシステムの境界を設定するための基盤として使用します[†6]。つまり、コンテキスト境界から、ビジネスアーキテクチャやビジネスケイパビリティへと話を進めることができます。

[†6] “The Death of Competition”（Harper）の著者であるJames Moore氏は、ビジネスエコシステムを「顧客のニーズを満たすために協力的かつ競争的に働く企業の集合体」と定義しています。

2.3.2.1　ビジネスケイパビリティ

ビジネス上の課題を解決し、ニーズを満たすためには、**ビジネスケイパビリティ**を使用するのが一般的です。ビジネスケイパビリティは、ビジネスアーキテ

クチャ内で使われる構成要素です。この言葉は、Ulrich Homann氏によって初めて使われたものであり（https://oreil.ly/JhjPW）[†7]、次のように定義されています。

> *ビジネスケイパビリティとは、「特定の目的や成果を達成するために、ビジネスが所有または交換できる特定の能力のこと」です。例えば、従業員の給与支払いや製品の出荷など、顧客に価値を生み出すためにビジネスが行うこと（成果やサービスレベル）を表したものです。ビジネスケイパビリティは、人、プロセス／手順、テクノロジー、情報を抽象化し、重要な構成要素としてカプセル化することで、性能改善や再設計のための分析を容易にします。*

[†7] Microservices.ioの著者の一人であるChris Richardson氏も、ビジネスケイパビリティという考え方について言及しています（https://oreil.ly/wBZj2）。

ビジネスケイパビリティが重要なのは、企業が戦略的なビジネスゴールと目標を達成するために、ビジネスの実態を抽象化したものだからです。ビジネスケイパビリティは、データや、プロセス、人材、テクノロジー間の関係を特定のコンテキストで表現し、エンドツーエンドの価値を提供するために活用されます。ビジネスケイパビリティが実際にどのように使われるかを明らかにするために、まずバリューストリームについて説明しましょう。**バリューストリーム**は、その名の通り、付加価値に焦点を当てたものです。バリューストリームは、一般的には、価値を提供するために行われる一連の行動のことです。例えば、製品や、サービス、エクスペリエンスを顧客に提供することもバリューストリームです。

バリューストリームを切り口として考えれば、それぞれのステップで価値を提供するためにはビジネスケイパビリティが必要となることがわかります。**図2-6**は、バリューストリーム、ビジネスケイパビリティ、およびその根底にあるビジネスプロセス、テクノロジー、情報の関係を示しています。バリューストリームとビジネスケイパビリティの関係は多対多です。例えば、**図2-6**では、企業間（B2B）のバリューストリームが複数のビジネスケイパビリティに関連付けられています。これらのビジネスケイパビリティの一部は、消費者向けビジネスのバリューストリームでも使用されます。

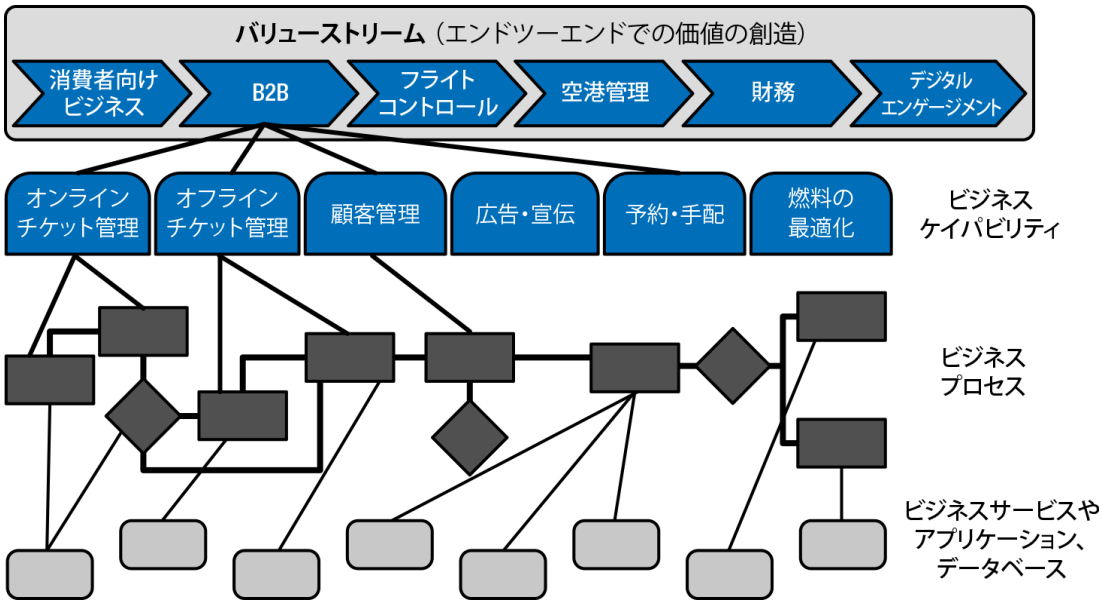

図2-6　バリューストリームをビジネスケイパビリティにマッピングする

複雑さに対処するには、まず初めに全体像を把握します。この点において**ビジネスケイパビリティモデル**は有用です。例えば、どのようなアプリケーションを共有するかを決めるのに必要なインサイトを提供してくれます。ビジネスケイパビリティモデルは、組織全体の戦略的なビジネス目標と作業を構造的に表したものです。それぞれのビジネスケイパビリティは、バリューストリーム内で少なくとも一度は実装され、使用されます。ビジネスケイパビリティは、通常、ビジネスが何を行うかを階層的に記述した**ケイパビリティマップ**で表現されます。一般的には、レベル1、レベル2、レベル3というビジネスケイパビリティ（ビジネスの複雑さによっては、さらに多くのレベル）に分けて考えます。ここで、それぞれのレベルは、上位レベルのケイパビリティを細分化したものになります。

ビジネスケイパビリティモデルには設計原則があります。ビジネスケイパビリティはそれぞれ一意であり、相互に排他的であり、かつ全体としては網羅しており、重複や重なり、隙間がないようにしておかなければなりません。

ビジネスケイパビリティモデルは、ビジネスと関連し、ビジネス用語で定義されていることが重要です。ビジネスケイパビリティの持つ意味は、具体的なビジネス目標や評価指標と関連付けることができます。ビジネスケイパビリティモデルの表すビジネスの概観は安定しておりほぼ変化しません。なぜなら、基本的なビジネス目標が変化することはほとんどないからです。ビジネスケイパビリティモデルは、バリューストリームやアプリケーション、プロセスへのマッピングや依存関係の把握にも活用できます。また、新たな戦略がビジネスケイパビリティにどのようなインパクトを与えるかを検討する際にも使えます。例えば、データマネジメント戦略の有効性と組織へのインパクトを示すために、ビジネスケイパビリティにKPIを設定することもできます。

図2-7に、架空の航空会社であるオーシャニック航空のビジネスケイパビリティマップを作成しました。オーシャニック航空が事業を拡大するためには、ここに挙げたすべての機能を使いこなす必要があります[†8]。例えば、オーシャニック航空は、航空券の販売、手荷物や遺失物の処理、航空機のメンテナンスができなければなりません。オーチャニック航空は、ある業務をアウトソーシングしたり、ある業務をビジネスの中核に位置づけることもできます[†9]。

[†8] オーシャニック航空は、映画、テレビ番組、漫画などでよく使われている架空の航空会社の名前です。例えば、テレビドラマ「LOST」ではこの航空会社の名前が使われています。

[†9] ビジネスケイパビリティは、組織構造や組織の境界については何も気にしません。また、ビジネスケイパビリティが、社内で開発されたものか、アウトソーシングされたものかも問題ではありません。ビジネスケイパビリティは、組織構造とは無関係に、組織内や組織間で移動することもあります。例えば、オンライン決済サービスを考えてみましょう。最初は社内で開発され、後に PayPalにアウトソーシングされる場合もあります。このような組織上の変更が、ビジネスケイパビリティに影響を与えるべきではありません。企業の目標を達成するためにオンライン決済を行う、という機能にはなにも変わりがないからです。

図2-7　航空会社の業務ドメイン分類例。このビジネスケイパビリティモデルは、ビジネスケイパビリティの観点から組織を高レベルで分解したもの

実際に目にするのは、ほとんどの社員がこれらのケイパビリティを中心に組織化されているということです。同じビジネスケイパビリティに携わる人々は、同じ語彙を使います。アプリケーションやプロセスについても同様です。一般的に、これらのアプリケーションやプロセスは、サポートする業務の凝集度に基づいて、適切に連携され、密に結合されています。このように、ビジネスケイパビリティマップは出発点として最適です。しかし、ここで話は終わりません。

2.3.2.2　ビジネスケイパビリティとアプリケーションの関連付け

エンタープライズアーキテクチャをうまく管理するには、ビジネスケイパビリティ、コンテキスト境界、アプリケーションが整合している必要があります。その際、いくつかの基本ルールに従うことが大切です。ビジネスケイパビリティは、ビジネスレベルでの定義にとどめ、抽象的にしておくことが重要です。ビジネスケイパビリティは、組織が行うことを表し、**問題空間**を対象とします。ビジネスケイパビリティは、コミュニケーションや戦略のための共通言語となります。

ビジネスケイパビリティが実装されると、特定のコンテキストに対して具現化したもの、つまり、「ケイパビリティインスタンス」が作成されます。この**解決空間**の境界内では、複数のアプリケーションやコンポーネントが連携して、特定のビジネス価値を提供します。ある特定のビジネスケイパビリティ用のアプリケーションやコンポーネントは、ほかのビジネスケイパビリティ用のアプリケーションからは分離されます

ここで重要なのは、論理的なアプリケーション境界を決めてしまうことです。理想的には、アプリケーションは、1つのケイパビリティインスタンスにのみ関連付けられます。複数のケイパビリティインスタンスにまたがったり、同時に関連付けたりするべきではありません。ケイパビリティインスタンスの境界内にある、これらのアプリケーションのセット全体が、コンテキスト境界となります。データやプロセスについても同様で、これらもまとめられます。

アプリケーション境界

アプリケーションのスコープと境界はどのように定義されるのでしょうか。**アプリケーション**という言葉の意味が人によって異なるので、アプリケーションの境界線の描き方は、その人の考え方によって異なります。例えば、アプリケーションのコードやデータをどのようにまとめるのか、どのようなグループで一緒に配備するのか、ほかのアプリケーションのコードなどからどのように隔離するのかなどの考え方で定義する方法があります。また、プロセスの実行という観点で見れば、アプリケーションは仮想マシンなどの仮想アドレス空間と考えることもできます。プロセスが、この仮想アドレス空間で実行されるからです。さらに、アプリケーションを、プログラム言語のランタイムシステム（https://oreil.ly/NFQjS）を共有し、サービスレイヤーを介して、ほかのコードと対話するアプリケーションコンポーネントの集合体と見なすこともできます。

ビジネスの観点からは、アプリケーションとは、特定のビジネスニーズを満たすために一緒に使用される、すべてのコードとデータのことと考えることができます。このような視点であれば、物理環境や、実行環境、プラットフォームが異なっていても、これらの環境にまたがってアプリケーション境界を設けることができます。また、アプリケーション境界を、企業内の（政治的な）オーナーシップや組織構造に基づいて設定することもあります。まとめると、アプ

リケーションとは何かについての答えは、読者がどのような立場でアプリケーションについて考えるかで大きく変わります。

データ管理に関しては、2つの視点を組み合わせて、ブレイクダウンしていきます。アプリケーションの概念自体は抽象的なものなので、アプリケーションは論理的な境界となります。これに対して、アプリケーションコンポーネントは、この境界の中で組み合わせて構築・配備されるものです。アプリケーションコンポーネントは、アプリケーションとは異なり、具体的なものです。アプリケーションコンポーネントは、解決空間が対象です。大きさや構成を持ちます。テクノロジーを使って作られます。具体的には、DockerコンテナやJARファイルとして、パッケージ化され配備されるアプリケーションモジュールなどのことです。

この2つの異なる視点を組み合わせることで、多くの価値が生まれます。アプリケーションレベルでは、ビジネスケイパビリティ、プロダクトオーナー、DevOpsチーム、プロジェクトなどを結び付けることができます。アプリケーションコンポーネントレベルでは、IT製品、バージョン、構成アイテム[†10]、契約、保守計画などを結び付けることができます。

[†10] ITIL（Information Technology Infrastructure Library）の用語では、構成アイテム（CI）とは、構成管理の対象となるインフラストラクチャーの構成要素のことです。

ビジネスケイパビリティとアプリケーションアーキテクチャの関係を明確にすることで、多くのメリットが得られます。具体的には、インタフェースとアプリケーションを分離するためのガイドラインが明確になります。データガバナンスを、効果的に実施できるようになります。例えば、ある特定のビジネスケイパビリティのコンテキストで作成されたすべてのデータは、ある特定のプロダクトオーナーに属すると断言できます。プロダクトオーナーシップとデータオーナーシップは、これまでよりはるかに整合性が取れるようになります。また、ケイパビリティレベルについても、簡単に結論を出すことができます。アプリケーションがケイパビリティに明示的に関連付けられているからです。

ドメイン境界とグラニュラリティ（粒度）

境界やグラニュラリティ（粒度）をどう設定するかは、実際には科学的ではありません。それは芸術的なものです。そして、ヒューリスティックス（経験則）を伴うものです。筆者は、ドメイン境界をビジネスアーキテクチャの論理的な境界にマッピングするようにしています。このようにすると、データ管理や企業の規模と複雑さを理解できるからです。組織の境界や、ビジネスプロセス、ドメインの専門性に注目して境界を設定する方々もいます。このほかにも、ドメイン境界の考え方として、ソフトウェアアーキテクチャの詳細なブループリントを作成し、どのアプリケーションコンポーネントを、より疎結合にするか、より密結合にするかを検討するという考え方もあります。この手法は、特にマイクロサービスの中では比較的人気のある手法です。ここで紹介した手法は、どれを使ってもかまいません。どれを適用するかは、課題を解決しようとしているビジネスのスコープや、規模、コンテキストに応じて決めてください。

ドメイン境界とは何か、どのようにしてコンテキスト境界を設定するのかを、よく理解したいのであれば、ドメインストーリーテリング（https://oreil.ly/Imivx）、イベントストーミング（https://oreil.ly/uNizW）、コンテキスト境界キャンバス（https://oreil.ly/cWXvT）をご一読ください。これらのテクニックを活用すれば、ドメインの複雑性を理解し、ドメイン内で何が起こっているのかがわかり、ドメインを構成したり、分解したりする方法を学ぶことができます。

ある特定のビジネスケイパビリティに対応するアプリケーションやコンポーネントは、ほかのビジネスケイパビリティに対応するアプリケーションやコンポーネントから分離されます。このようにすることで、柔軟性を高めることができ、前述した塀と家の話と同じように、ドメインとコンテキスト境界が機能します。コンテキスト境界は、ビジネスケイパビリティから生まれ、ビジネスケイパビリティに排他的にマッピングされる、という原則を明確に設定できます。コンテキスト境界は、ビジネスケイパビリティの実装の境界を表し、ドメインのように振る舞います。ビジネスケイパビリティが変更されると、コンテキスト境界も変わります。本来であれば、ドメインと、対応するコンテキスト境界は完全に一致していることが望ましいのですが、「2.3.3　ドメインの特性」で説明するように、実際には異なる場合があります。

もっと具体的に説明するために、オーシャニック航空の例を見てみましょう。**図2-8**は、この架空の会社で、コンテキスト境界とドメイン実装の一部がどの

ようになるかを示しています。

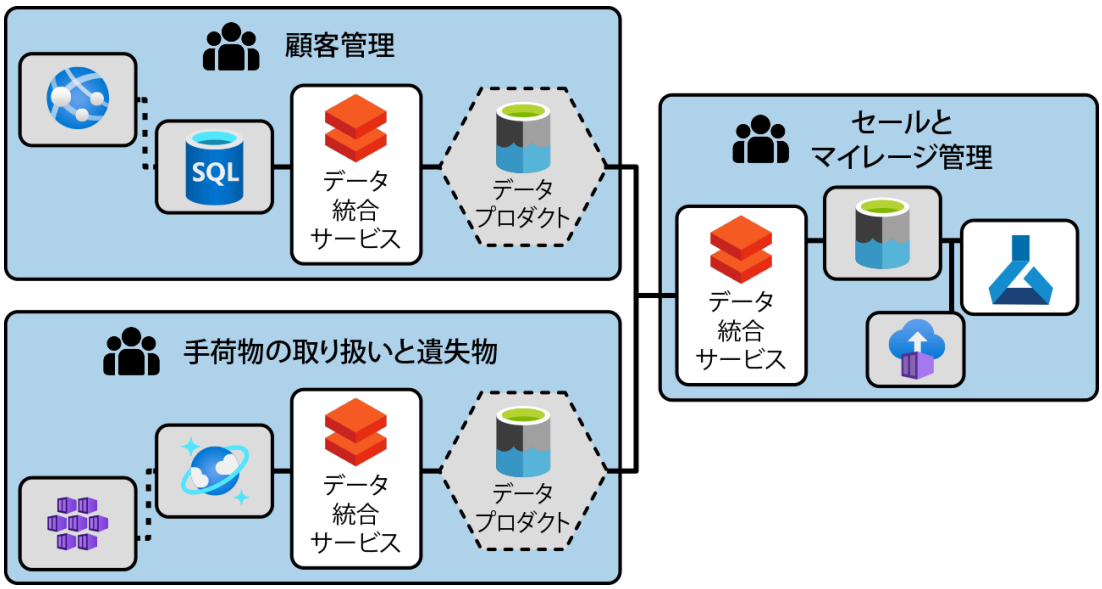

図2-8　アプリケーションとコンポーネントが解決空間でどのように連携するか

顧客管理ドメインは、顧客とその顧客との取引関係を管理しなければなりません。このドメインを管理するチームは高度な専門知識を持っています。関連するデータとしてどのようなものが存在するかを知っています。したがって、どのデータをほかのドメインに提供すべきかも熟知しています。このドメインの内部アーキテクチャは分離されているため、境界内のアプリケーションコンポーネントはすべて、アプリケーション独自のインタフェースとデータモデルを使って直接通信することができます。この手法では、データはドメインのコンテキストと整合性が保たれます。そのため、データは自動的にユビキタス言語を継承します。ユビキタス言語とは、同じドメインのステークホルダーや設計者間で合意され、そのドメインのニーズに合わせて構築され、形式化された言語です[†11]。ほかのドメインへのデータ流通は形式化されます。共通の通路を通してデータを外に出し、その通路のルールに従わなければなりません。つまり、ドメインがほかのドメインと通信する際には、標準化されたインタフェースを使用することになります。

[†11] 言語を形式化する方法として一般的なのは、すべてのビジネス用語と定義をデータカタログに格納することです。

2.3.2.3　ケイパビリティの実現

ビジネスケイパビリティマップを使用する場合、ビジネスケイパビリティの中には複数回インスタンス化されるものがあることを理解しておいてください。図2-9に、ビジネスケイパビリティとビジネスケイパビリティインスタンスの関係を示します。

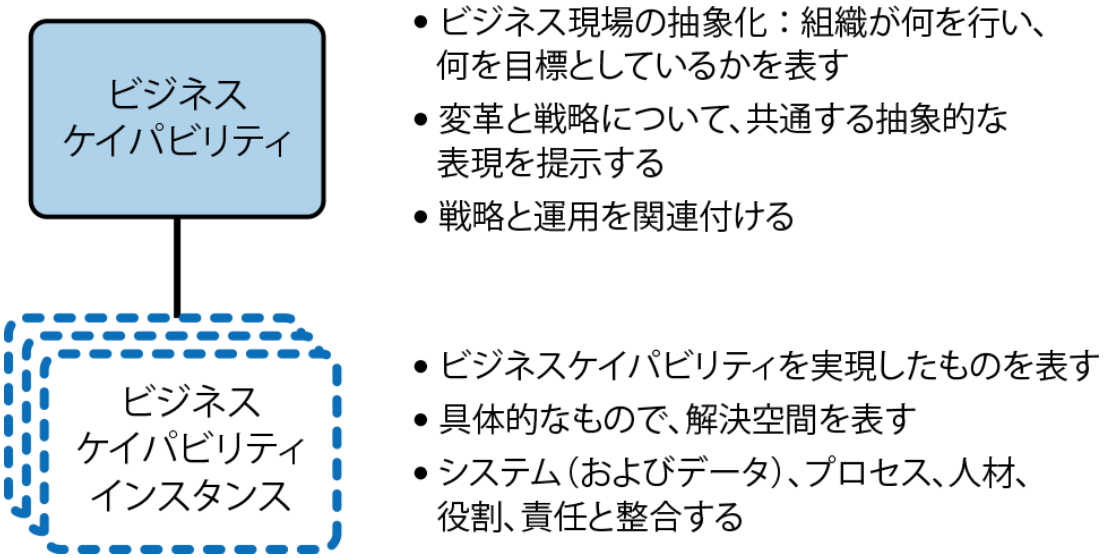

図2-9　ビジネスケイパビリティとビジネスケイパビリティインスタンスの関係は、1対多

例えば、オーシャニック航空は、「手荷物の取り扱いと遺失物」の機能を、アジア、ヨーロッパなどの地域ごとに実現してもかまいません。これらの実装は、それぞれ異なるテクノロジーを使用し、異なるチームがそれらのサービスを運用する可能性もあります。ビジネスケイパビリティとケイパビリティインスタンス（実装）の関係は1対多であるため、ビジネスケイパビリティを何度か実装していくと、（サブ）ドメインを追加することになるでしょう。

2.3.2.4　共有ケイパビリティ

さらに重要なのは、共有ビジネスケイパビリティをどのように管理するかです。このようなケイパビリティは通常、サービス（as-a-service）モデルを使って中央で実装され、さまざまな事業部門に提供されます。「顧客管理」は、そのようなケイパビリティの一例です。オーシャニック航空のアジアとヨーロッパの事業部門は、顧客管理に同じシステムを使うかもしれません。では、どのようにして共有ケイパビリティのドメインデータオーナーシップを確立すればよいのでしょうか。共有管理システムとデータベースという同一のシステムに格納されている顧客データに対して、複数のビジネス担当者が責任を持つことになるはずです。結論から言えば、アプリケーションドメインとデータドメインがあるということです。データの観点からは、ドメインとアプリケーショ

ンのコンテキスト境界は常に完全に一致するわけではありません。逆に、ビジネスケイパビリティの観点からは、データとアプリケーションの関心事は同じになるでしょう。

企業資源計画（ERP）、サプライ・チェーン管理（SCM）、顧客関係管理（CRM）システムのモノリス型の性質のために、組織はほぼ間違いなくビジネスケイパビリティを共有することになります。これまでのところ、これらのシステムは多くのドメインにまたがっています。そのため、早い段階でこの問題を解決する必要があります。

ベンダーの複雑なパッケージ、SaaSソリューション、レガシーシステムなどの共有機能については、ドメインデータオーナーシップに一貫性を持たせるのがお勧めです。1つの方法として、データプロダクトによってデータオーナーシップを区別するという方法があります。これは、同じアプリケーションドメインから、さまざまなデータドメインに対応する複数のデータプロダクトを生成するということです。顧客管理の例で説明すると、アジア関連の顧客にはアジア関連の顧客用のデータプロダクト、ヨーロッパ関連の顧客にはヨーロッパ関連の顧客用のデータプロダクトというように、異なるデータプロダクトを使用して異なる関心事を表します。ほかの方法として、アプリケーションドメインに単一のデータプロダクトを設計し、そのデータプロダクトの内部でデータオーナーシップを識別するためのメタデータをカプセル化するという方法もあります。例えば、オーナーシップのために印を付けた列名を使用することで、それぞれの行を1つの特定のデータドメインにマッピングすることができます。8章では、データガバナンスとセキュリティについて、図解と補足説明を行います。

図2-10は、汎用ビジネスケイパビリティと共有ビジネスケイパビリティの違いを示しています。ご覧のように、共有ケイパビリティが複雑になっているのは、さまざまなドメインがそのケイパビリティを使用するにもかかわらず、ケイパビリティの実装を提供するプロバイダーが1つしかないためです。

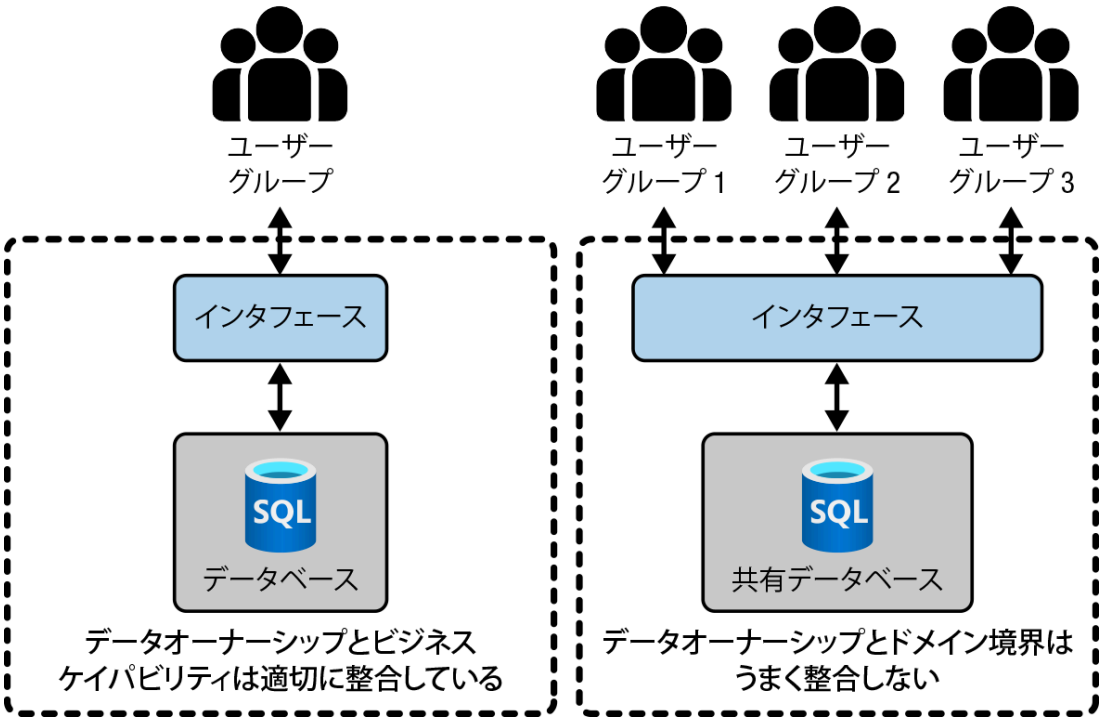

図2-10　汎用ビジネスケイパビリティ（左）と共有ビジネスケイパビリティ（右）

理想的には、ビジネスケイパビリティを所有するチームはアプリケーションのドメインと整合しているため、ソフトウェアオーナーシップとデータオーナーシップも整合します。ドメインを分割できない場合は、アプリケーションのオーナーシップとデータのオーナーシップを分ける必要があります。例えば、顧客管理の例では、テクノロジーサービス、パイプライン、アプリケーションコンポーネントに責任を持つ1人のアプリケーションオーナーと、メタデータの登録、プライバシー、セキュリティなど、さまざまなデータセットに責任を持つ複数のデータオーナー（ビジネス担当者）がいることになります。

2.3.2.5　複雑なアプリケーション

また、大企業や 昔ながらの企業にありがちな、複数の業務に対応するアプリケーションにも気を配る必要があります。例えば、オーシャニック航空は、コスト管理と資産・リースの両方に同じソフトウェアパッケージを使うかもしれません。このようなアプリケーションは、一般的に規模が大きく複雑なものとなります。ベンダーのビジネススイートであったり、できるかぎり多くの機能を持つ古いモノリス型のものであったりします。これは、前の項で説明した問題と似ているように見えるかもしれませんが、この場合にはその複雑さが違っています。1つのビジネスケイパビリティがチーム間で共有されているのではなく、1つのアプリケーションとデータベースが異なるビジネスケイパビリティに

よって共有されているのです。この場合、前述したのと同じように、データオーナーシップの問題が発生します。

2.3.3　ドメインの特性

ドメインとアプリケーションをマッピングする際、データの作成、消費、再配信を考慮すると、さまざまなデータ流通パターンがあることがわかると思います。原則を定義し、アーキテクチャのブループリントを設計し、これらをドメインの特性に整合させることをお勧めします。抽象的なレベルでは、ドメインを2種類に分類することができます。

ソースシステム整合ドメイン（Source system-aligned domain）は、データの発生源であるゴールデンソースシステムに整合したドメインです。これらのシステムは典型的にはトランザクションシステムや運用システムです。これらのドメインからデータを共有する場合の原則は、ゴールデンソースシステムから直接データを取得する、ということです。データとオーナーシップ情報を正しく取り込むには、データ変換と共有を標準サービスでサポートする必要があります。この種類のドメインの例を図2-11に示します。このドメインでは、データプロダクトをほかの消費ドメインに流通させるときに「共通ドライブウェイ」パターンを使用します。

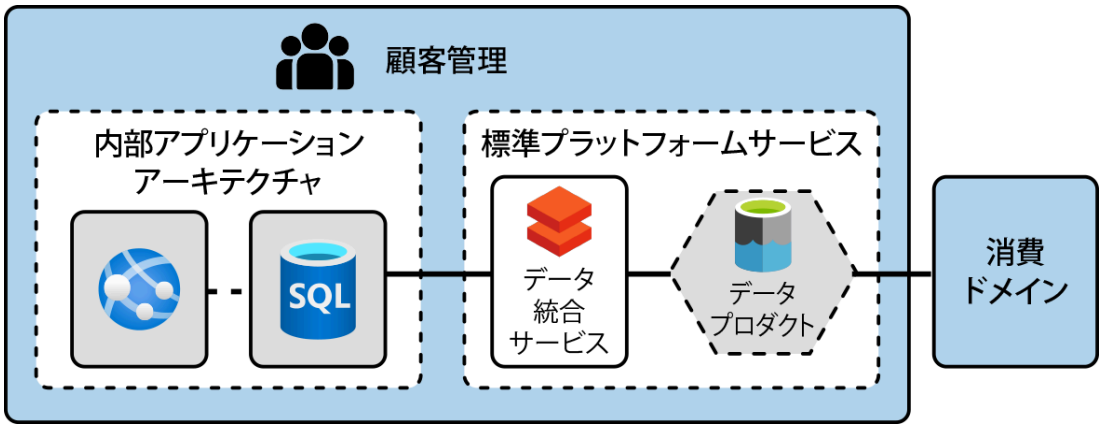

図2-11　顧客管理ドメインはソースシステム整合

コンシューマ整合ドメイン（Consumer-aligned domain）は、ほかのドメインからデータを消費して利用するドメインです。このドメインは、ほかのドメインのデータを必要とする、ある特定のエンドユーザーのユースケースと整合するので、ソースシステム整合ドメインとは正反対のものです。エンドユーザーは、ビジネスニーズに合わせてデータを消費し、変換します。このような消費ニーズに対応するために、標準サービスを提供して、データ変換と消費を行

うドメインをサポートする必要があります。消費ドメインの例を図2-12に示します。このドメインでは、データを価値に変換するために標準プラットフォームサービスを使用しています。

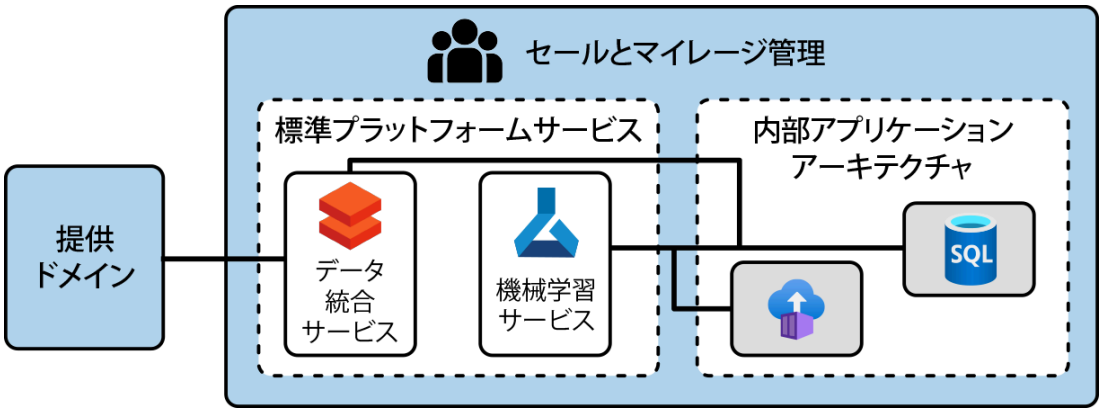

図2-12　セールとマイレージ管理ドメインは、コンシューマ整合ドメイン

10章で詳しく説明しますが、もっと複雑なケースは、データを再利用する必要がある場合です。複数のコンシューマがさまざまなドメインのデータを同じように組み合わせて利用している状況を考えてみてください。このような複数のコンシューマが共有する関心事を効率的に管理したい場合、データが使用される場所とは別の場所でビジネスロジックを構築して、共有データをまとめて作成したくなるでしょう。図2-13に、実際にこのような仕組みを示します。この例では、「排出権と賦課金」ドメインが、まずデータを結合・統合します。次にこのデータを自分自身で使用し、それをほかのドメインと共有することで、集約ドメインのように機能します。このカスタマー・サプライヤーパターンは、ドメイン間の統合要件が重なる場合に特に有効です。

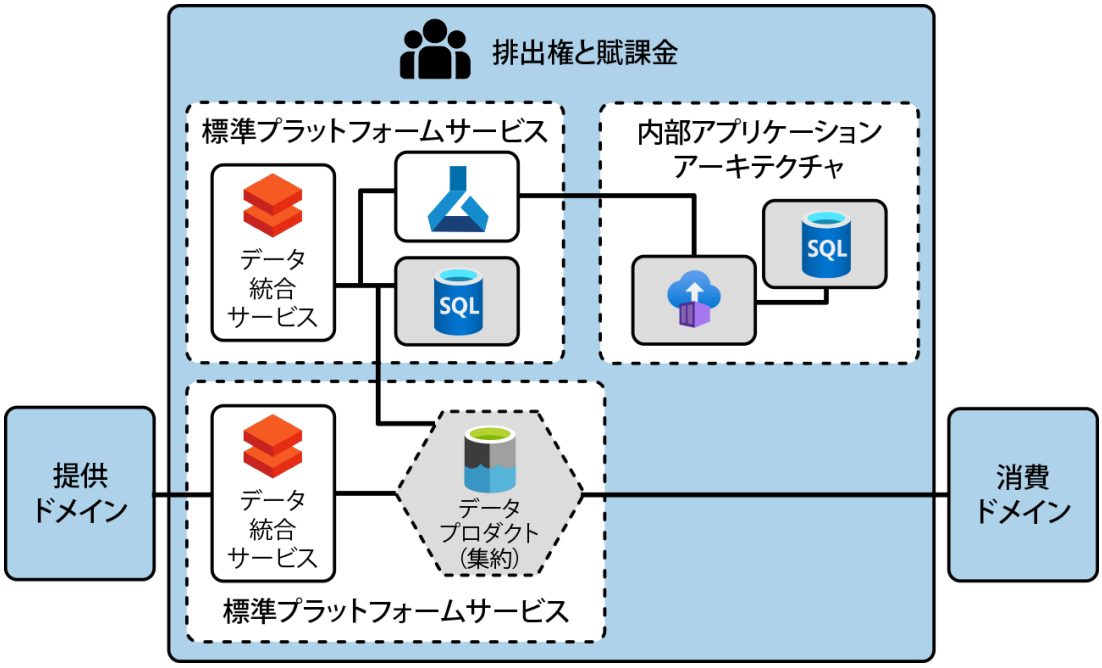

図2-13　排出権と賦課金ドメインでは、さまざまなパターンが混在する

また、純粋に上流・下流の関係だけでドメインを連携させるというコラボレーションモデルもあります。この場合、**集約ドメイン**は、ほかのドメインに結合データを提供するための専用サプライヤーとして機能します。このようなサプライヤードメインは、データコンシューマでもあり、データプロバイダーでもあります。このサプライヤードメインは、データを消費し、ドメイン集約（結合データプロダクト）を行い、それを直接流通します。このコラボレーションモデルでは、新たに作成されたデータプロダクトはドメイン自身のユースケースには使われません。

ドメイン集約は、1つの要素として取り扱えるドメインオブジェクトの集合体です。同じ形式や型のオブジェクトの集まりを一緒に使用する場合、それらを1つのオブジェクトとしてモデル化することができます。このようにすることで、ほかのドメインでそれらを簡単に使用することができるようになります。なお、ドメイン集約と、データをグループ化して要約して表現するデータ集約とを混同しないでください。

また、図2-14に示すように、コンシューマが新しいデータを作成し、プロバイダーになる場合もあります。例を挙げれば、乗客の旅行傾向について予測を行う場合などです。消費ドメインは、まず、ほかのドメインのドメインロジックの一部を自分のドメインロジックと組み合わせて使用します。そして、集約されたデータを、ほかのドメインでも利用できるような新しいデータプロダクトに変換し、プロバイダーとして機能します。このようなドメインは、新たな何かを生成するので、**コンストラクタドメイン**と呼ばれることもあります。

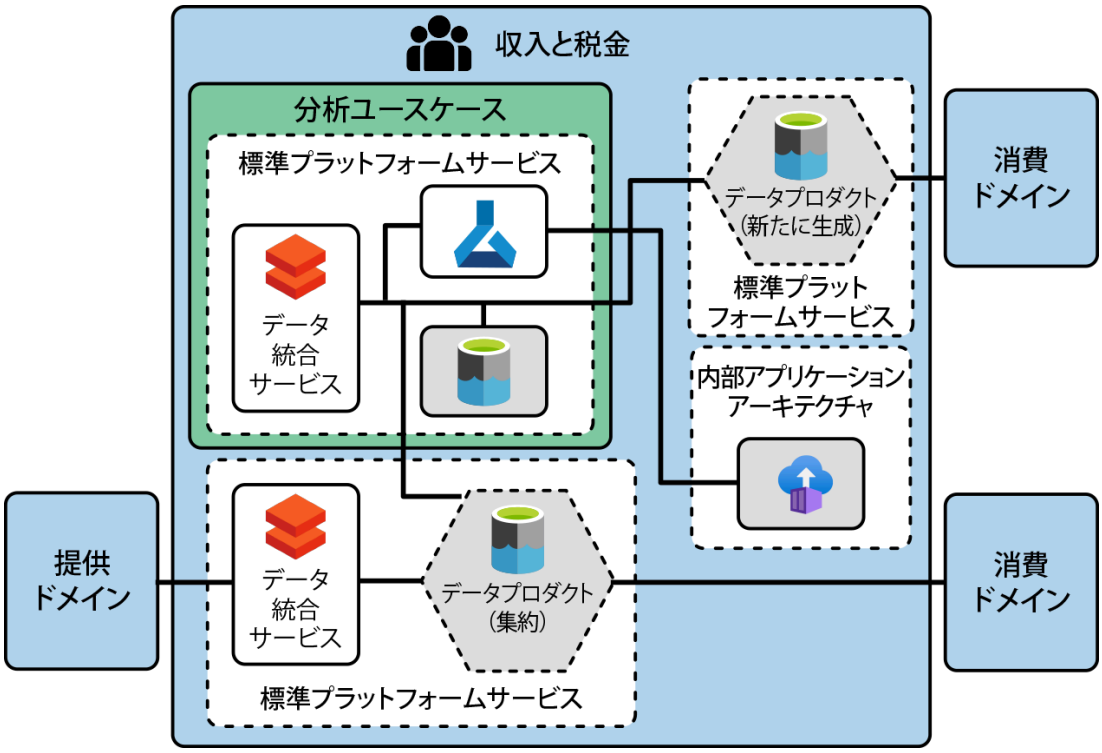

図2-14 「収入と税金」ドメインはコンストラクタドメインとして機能する。データを集約して自身の目的に使用し、新しく作成されたデータプロダクトをほかのドメインにエクスポートする

先ほどのドメイン集約と異なる点は、新しく生成されたデータプロダクトには新しいビジネスインサイトが含まれている点です。コンシューマに送られるデータは、構造が変わっただけでなく、異なるコンテキストを使って新しいビジネスインサイトが追加されたことで、内容がリッチ化されています。

これらすべての場合において、データを流通する際には共通ドライブウェイを使用するというベストプラクティスに従ってください。

これまで見てきたように、データプロダクトの使い方や流通方法の特性によって、ドメインを分類することができます。場合によっては、ビジネスのルールが変わることもあります。例えば、ブループリントやアーキテクチャパターンが変わってくる場合もあります。以降の章では、今説明した一連のシナリオについて、再び説明します。

2.3.3.1　複雑な統合の課題に対応するためのパターン

企業全体をより細かいドメイン構造に分解すると、複雑な統合の課題が発生します。オーシャニック航空を例にとれば、「顧客管理」ドメインの顧客データはさまざまな多くのドメインで必要とされる可能性があり、「運航計画管理」ドメインは、運航計画を立てる前に、パイロットが「パイロット管理」ドメインの管理システムに存在しているかどうかを確認しなければなりません。このような複雑な問題にどのように対処すればよいのでしょうか。

ベストプラクティスは、いくつかの共通ドライブウェイパターンを標準化し、さまざまな形式の統合を実現する方法です。例えば、大規模なデータ処理を行う場合、ドメインはほかのドメインにデータプロダクトを提供し、集中的にデータを読み込むことができるようにします。例えば、強一貫性のある読み出し（およびコマンド）には、APIサービスを提供します。これらに関連するトピックについては、4章、5章、6章で詳しく説明します。

2.3.3.2　ビジネスケイパビリティモデルの強み

ビジネスケイパビリティモデルのメリットは、分散アーキテクチャにおいてドメインをより適切に把握し、整理できるようになることです。データとアプリケーションがビジネスにどのような価値をもたらすかを全体的に把握することができるようになります。その結果、データ戦略と真のビジネスニーズに優先順位をつけて注力できるようになります。また、このモデルは、データ領域以外でも活用できます。例えば、スケーラビリティが懸念される場合、最も重要なコアビジネス機能を特定し、それに対する戦略を策定することができます。

このようなターゲットとなる状態のアーキテクチャを、すべてを前もってマッピングして構築することは、作業量が多すぎると懸念を抱く人もいるでしょう。そこで提案するのが、新しいアーキテクチャに取り組みながら、ドメインを洗い出していく方法です。この方法では、トップダウンでターゲット状態を定義するのではなく、ボトムアップで現在の状態をターゲット状態へと移行させるとともに、調査、分析、実験を行っていきます。ただし、この方法では、当初は時間を節約できるかもしれませんが、事態が悪化し始めたときに、複雑な移行処理や修正作業が必要になる恐れがあります。よりバランスの取れた手法としては、大まかに全体像を描き、時間をかけて細部を描き足していくという、両方向から取り組んでいく手法です。

エンタープライズデータ管理アーキテクチャを運用するには、パターンを標準化し、明確な設計原則に基づいて作業を行い、難しい選択をしなければなりません。次の節では 基本的な設計の選択から始めて、徐々に非常に重要なパターンと原則を紹介していきます。

2.4　分散型ドメイン指向データ管理の原則

ここまでのところで、データ共有モデルとクロスドメインでのコラボレーションモデルについて、はっきりとご理解いただけたと思います。これから、分散データ管理戦略が成功するために守るべき重要な原則をいくつか紹介することにしましょう。まずは上位レベルのアドバイスから始めて、データドメインやデータプロバイダーに関する具体的な話に入ります。

データのサイロ化を避けます。
最初の設計原則は、データ流通にサイロや統合ハブを使わないことです。データサイロでは変換ステップが必要になるため、プロバイダーとコンシューマ間のチェーンが長くなり、管理に手間がかかります。例えば、データウェアハウスモデルでは、すべてのデータを統合する際に、コンテキスト境界が衝突することが多いため、不整合に対処しなければなりません。また、データウェアハウスモデルでは、データプロバイダーとデータコンシューマが密結合しているため、依存関係の管理が非常に難しくなります。多くの場合、データ品質の問題、修正、不整合のために、データサイロを信頼できなくなります。
データ流通のためのデータサイロを排除し、最新のデータ統合と流通の形態を使用することで、統合から生じる問題を回避し、管理が困難なポイントツーポイント統合アーキテクチャを使わずにすむようになります。

ポイントツーポイントのインタフェースを使用してドメインを接続すると、指数関数的に複雑になるので絶対に避けるべきです。例えば、100個のアプリケーションを相互接続すると、5,000のポイントツーポイント接続が生じます。1,000個なら、およそ500,000です。ポイントツーポイント接続では、通信チャネルの数が膨大になるため、すべての依存関係を把握することはほぼ不可能となり、企業が必要とするコントロールとアジリティを確保することができません。

データはゴールデンソースでのみ取得し、修正します。

2つ目の原則は、データに対する信頼を高めることを目的としたものです。ほかのシステムにデータを流通させる前に、必ずゴールデンソースから一意のデータのみを取得するようにしてください。元データを変更する場合は、オーナーの承認を得て、ドメイン内のゴールデンソースに対してのみ変更するようにします。透明性と信頼のために、ゴールデンソースを周知しておくことも重要です。そのためには、すべての一意のゴールデンソースシステムを管理する中央レジスタやカタログを設けることをお勧めします。

データオーナーシップのルールを尊重します。

アプリケーションがデータを消費し使用するとき、新たなゴールデンソースとなり得ます。データは新しいコンテキストを持つことになり、新しいファクト（事実）が生まれます。データが変更され、新しいファクトが作成されると、**新しいオーナーシップが生じることになります**。なお、フィルター、結合、大文字から小文字への変換、フィールド名の変更といった単純な「構文上の変換」は、すべてのファクトが同じままですので、新しいオーナーシップは必要ありません。この原則は、もう一つの基本原則である「**自分が所有していないデータは流通させない**」ということと強く関連しています。ほかのドメインに由来し、ほかのドメインが所有するデータを流通させるときは、必ず自分のドメインがそのドメインから承認を得るようにしてください。

2.4.1　データドメインの設計原則

ドメイン駆動設計を企業規模で実施する際に、依存関係の管理と関心事の分離において大きく変化するところは、DDDの哲学とコンテキスト境界を使ってアプリケーション全体の論理的境界を設定する点です。コンテキスト境界は、先ほど説明したように、問題領域を適切に管理するために、適切な凝集度と理想的な大きさを見つけて設定します。一旦コンテキスト境界が決定されると、それは分離され、ほかのコンテキスト境界と通信する際には形式化されたエンドポイントを使わなければなりません（これが、これまで議論してきた共通ドライブウェイパターンです）。

企業レベルでドメイン駆動設計モデルを使用することは、サイロ型を使用することとはまったく異なります。サイロ型では、すべてのデータが、エンタープライズ言語を使って中央集権化され完全に統合されています。これに対して、DDDでは、コンテキスト境界ごとに独自のユビキタス言語を使います。コンテキスト境界が変われば、ユビキタス言語も変わります。新しいアーキテクチャにおけるエンタープライズ言語は、多くのユビキタス言語の集合体となります。

DDDモデルでは、データ管理の中でどのようにまとまりを見つけて、どのようにコンテキスト境界を決定するかについては、明示されていません。そのため、新しいアーキテクチャを設計する際には、以下の設計原則にも従うことをお勧めします。

データはそれぞれのドメインで管理・配信されます。
データ品質、データパイプライン、データの可読性は、ドメインの関心事です。そのデータについて熟知しているはずの人々が、データの管理に責任を持ちます。データオーナーシップモデルは、中央管理ではなく分散管理となります。

コンテキスト境界は、インスタンス化されたビジネスケイパビリティに関係付けられます。
コンテキスト境界は、ビジネスケイパビリティインスタンスまたはその一部を実装します。つまり、コンテキスト境界は、その実装を表すものです。そのため、コンテキスト境界は同じ解決空間に焦点を合わせます。

コンテキスト境界は、1つまたは複数のアプリケーションに関連付けることができます。
コンテキスト境界は、1つのアプリケーションで構成されても、複数のアプリケーションで構成されてもかまいません。複数のアプリケーションで構成する場合、すべてのアプリケーションは、同一の（ビジネス）ケイパビリティインスタンスに対して、価値を提供することになります。

複雑なアーキテクチャを分解し、分離する方法はさまざまです。粒度と分離のレベルは、ドメインを使って調整することができます。ドメイン内の複数のアプリケーションは、コンテキスト境界を形成し、ほかのドメインから分離されます。また、粒度をより細かくして、アプリケーションとアプリケーションを分離するという方法もあります。これは、ある程度の自律性を持つドメインに対し、常にアプリケーション間を分離するように強制することになります。

ユビキタス言語は、コンテキスト境界の中で共有されます。

自身のコンテキスト境界内で、データを流通するアプリケーションやアプリケーションコンポーネントは、同一のユビキタス言語を使います。同じ用語や定義が別の意味で使われることはありません。それぞれのコンテキスト境界は、概念データモデルと1対1の関係になります。

コンテキスト境界は、インフラストラクチャー、ネットワーク、組織とは直接関係はありません。

アプリケーション機能、プロセス、データを踏まえて、どのようにまとめるかを考えてください。データ管理の観点で言えば、コンテキスト境界をどう作るかについては、インフラストラクチャーや、ネットワーク、組織は関係ありません。

1つのコンテキスト境界は、1つのチームに属します。

理想的には、それぞれのコンテキスト境界は、1つのアジャイルチームやDevOpsチームに属します。1つのチームにしか属していないのであれば、結合点は管理しやすくなり、チームメンバーも理解しやすくなるからです。

境界線は厳密にします。

コンテキスト境界の境界線は厳密にしてください。コンテキスト境界はそれぞれ区別して扱うようにしてください。一括りで考えるべきビジネス上の関心事や、プロセス、データは、一括りにして、コンテキスト境界内で保守・管理するようにしてください。

境界を越えるときは分離します。

コンテキスト境界内では、密結合できます。しかし、境界線を越えるときには、インタフェースで分離してください。

境界線についての原則を明確に守っているのであれば、あるコンテキスト境界でのデータ形式はどのようなものでもかまいません。

ドメインデータは、付加システムや中間システムを介して配信されるべきではありません。

クロスドメインでのデータ仮想化レイヤーなどの追加レイヤーは、ゴールデンソースシステムをわかりにくくします。このようなレイヤーは複雑さを増し、データとその品質を変化させ、データの出所がわかりにくくなります。データの受け渡しに付加システムや中間システムを使用することは推奨されません。

データメッシュアーキテクチャは、単一のデータモデルは存在しない、ということを強く主張しています。現実はそうではありません。というのも、データを正しくモデル化するためには、ビジネス全体で共有される関係を明らかにしなければならないからです。これらの関係を無視すると、もはやドメイン間で統合でき

ないデータになってしまいます。これらの点については、10章と11章で説明します。

汎用データサービスは、ドメインに依存しないものでなければなりません。
ドメインロジックや複雑な統合は、コンテキスト境界内にとどめるべきです。どの汎用データサービスにも置くべきではありません。こうすることで、ドメインは価値を提供することに集中できます。知らないところにドメインロジックが隠されているために、ほかのドメインとのインタラクションが損なわれてしまう、という不安を抱えなくてすむようになります。

DDDのアプローチでは、アジリティが大幅に向上します。データプロバイダーとデータコンシューマが疎結合になるからです。ドメイン間の依存関係はデータプロダクトなどのデータエンドポイントのみになるため、論理的なコンテキスト境界内に存在するビジネスドメインは、それぞれ独自の速度で進化できるようになります。

2.4.2　データプロバイダーのベストプラクティス

あるコンテキストから別のコンテキストにデータを移動させることは、両方のコンテキストに関する知識が必要となるため難しい作業となります。一般的に、アプリケーションはデータを大量消費できるようには作られていないため問題となります。スキーマは高度に正規化されていることが多く、ビジネスロジックがデータの中に隠されていたり、ドキュメントがなかったりします。データ統合の苦しみを和らげるには、あるコンテキスト境界から別のコンテキスト境界に対して、データをどのようにモデル化し公開・提示するかについて、いくつかの原則を守る必要があります。

アプリケーションの技術的な情報を隠します。
データコンシューマは、物理データモデルの専門家になる必要はなく、データを使用するためにアプリケーションの機能を再構築する必要もありません。アプリケーションの観点から見ると、データプロバイダー側は、アプリケーションロジックをすべて取り除かなければなりません。アプリケーションロジックは、アプリケーションに関する知識を必要とするからです。この原則は、マイグレーション情報や、ログ情報、データベーススキーマなど、アプリケーションでは一般的な技術データにもあてはまりま

す。このような技術データは、ドメイン内部でしか意味がありません。データプロバイダーは、データを公開する前に、データコンシューマが必要としないデータを除外する必要があります。

大量のデータ消費に最適化します。

多くのアプリケーションは、大量のデータを消費するようには最適化されていません。ドメインが簡単にデータを消費できるようにするためには、親子関係が延々と続く高度に正規化されたデータを、ほかのドメインに提供してはなりません。データは正規化しすぎず、直感的にグループ化されるべきです。ユーザーフレンドリーなフィールド名をつければ、データコンシューマ側で検索しやすくなります。命名規則やデータフォーマットに一貫性がなかったり異なったりしている場合は、事前に解決しておくべきです。このアプローチでは、データを汎用的に消費できるように最適化しなければならず、データコンシューマのニーズに応えられるように、可能な限り再利用できるようにしなければなりません。このことは、1章で述べた、書き込みに対して読み出しの比率が高まっていることとも関連しています。

ユビキタス言語は、通信のための言語です。

データプロバイダーとして機能するコンテキスト境界は、それぞれ独自のユビキタス言語を使ってデータを公開します。つまり、データプロバイダーは、ほかのドメインのビジネスロジックを自分のドメインに組み込むべきではありません。

インタフェースには一定の成熟度と安定性が求められます。

この原則は、変化を抽象化するためのものです。例えば、ドメインの扱う範囲が頻繁に変更される場合、その変化を抽象化して安定的なものにすることが求められます。また、スキーマも、ある程度安定した互換性を提供しなければなりません。

データはすべてのパターンで一貫しているべきです。

この原則は、さまざまなパターンでデータを公開する際に、その公開方法については一貫性を持つべきというものです。同じデータがさまざまな方法で公開される場合でも、例えば**顧客住所**という要素はすべての公開方法で一貫していなければなりません。

自己記述的でユーザーフレンドリーなデータを利用するという手法は、生データを直接共有するデータレイクの実装とは異なります。データレイク（「1.9.2　データレイク：構造化データと非構造化データに対する中央リポジトリ」参照）では、通常、データはソースから1対1のコピーとして、そのまま取り込まれ、その後（直接）下流で利用されます。データとインタフェースは、基盤となるソースシステムと密結合しているため、ソースシステムが変更されると、本番環境のパイプラインがすぐに壊れてしまいます。このようなことが起こると、企業はアドバンストアナリティクスを運用するのが難しくなってしまいます。このため、一般的なベストプラクティスとして、アプリケーションチーム

から修正の希望が出たとしても、その希望を受け入れてはなりません。その代わりに、ドメインチームには、物理データモデルをドメイン境界内にカプセル化するよう依頼しましょう。つまり、上流のドメインチームが独自のドメインオブジェクトやデータプロダクトを構築するようにします。

これらの原則から結論づけられるのは、データプロバイダーは、より使いやすく、消費しやすく、論理的な方法でデータを公開すべきだということです。生データに対して何度も作業を行わせるようなことは避けるべきです。データは、アプリケーションからの物理データモデルではなく、抽象化された論理ビジネスモデルとして提供されるべきです。4章では、良いデータ表現とはどういうものかについて、より具体的なガイダンスを示します。

2.4.3　ドメインオーナーシップの責務

ドメイン間でアーキテクチャのオーナーシップを連合型にするには、さらにいくつかの原則を守らなければなりません。ドメインがアプリケーションと、そのアプリケーションからのデータプロダクトのオーナーシップを持つことに加えて、ドメインチームは以下のような責任を負わなければなりません。

- データのインジェスト、クレンジング、変換など、データパイプラインのオーナーシップを持ち、できるだけ多くのデータカスタマーのニーズに応えます。
- データ品質を改善し、データコンシューマが設定したSLA（サービスレベルアグリーメント、Service Level Agreement）と品質指標を遵守します。
- 行/列レベルのきめ細かなフィルタリングと動的なデータマスキングを行うために、メタデータをカプセル化するか、予約列名を使用します。
- 以下のようなメタデータ管理標準を遵守します。
  - アプリケーションとソースシステムのスキーマ登録
  - ディスカバラビリティ（発見可能性）を高めるためにメタデータの提供
  - バージョン管理ルールの遵守
  - データ属性とビジネス用語の関連付け
  - ドメイン間の統合を改善するために、メタデータ情報の完全性の確保
- プロトコル、データフォーマット、データタイプなど、データ相互運用性の標準を遵守します。
- 手動で、またはソースシステムと統合サービスをスキャナーに接続することによって、リネージを提供します。
- IDやアクセス管理の確認やデータ契約書の作成など、データ共有に関する作業を遂行します。

以上の原則は、厳密に守っても、緩やかに守ってもかまいません。アドホックで、未知の実験的なユースケースの場合、インタフェースは不安定であっても、直接消費用に最適化されていなくてもかまいません。ただし、安定した本番ユースケースでは、原則に忠実に従うべきです。システム内で併用して、これらの手法を混在させることもできます。例えば、あるデータプロダクトに対してはここで説明した設計原則に厳密に従った方法で提供する一方で、別のデータプロダクト（おそらくそれほど重要でないもの）をより緩やかに提供することができます。

新しいアーキテクチャでは、データの流通と統合がさまざまな方法で行われるため、ここで挙げた原則を守ることが重要です。データ駆動型になり、データ消費を推進するには、すべてのドメインがデータとその提供方法を最優先事項として捉えなければなりません。ドメインはこれらの原則を遵守し、次の章から紹介していく、すべての通信パターンに適用しなければなりません。

2.5　分散型ドメイン指向データ管理への移行

ここまできたところで、読者は「これらのビジネスケイパビリティと原則は理にかなっているけど、何から始めたらいいのだろうか」と考えているのではないでしょうか。ドメイン指向のデータオーナーシップへの移行をうまく進めるために必要な条件は、以下のようにまとめられます。

- アプリケーション、アプリケーションコンポーネント、（データ）ドメイン、ビジネスケイパビリティなどの概念を説明するために、一連の用語について定義を行います。アプリケーションとドメインの境界をどのように引くかは、どのような視点に立つかによって異なります。これらの言葉の意味は、人によって異なるからです。したがって、明確な定義を作成し、これらの概念をどのように整合させるかを決めておくことが重要です。メタモデルの使用に関するガイダンスやサポートについては、9章で説明します。
- ビジネスケイパビリティマップを使って、ビジネス上の関心事を構造化します。大企業において、ビジネスケイパビリティは、組織を包括的に説明する素晴らしい方法です。ビジネスケイパビリティは、「何を」するのかということであり、「どのように」するのかということではないことに注意してください。実装の詳細とテクノロジーアーキテクチャとの整合性については、3章で説明します。

- ビジネスケイパビリティを明確にした後、どのようなアプリケーションとデータがどのようなビジネス上の関心事に寄与するかを具体的にマッピングします。ゴールデンソースを特定してリストアップし、データのオーナーを特定します。また、ITサービス管理ソリューションや、ServiceNow（https://oreil.ly/RzPgq）などの構成管理データベース、自作のアプリケーションカタログを使用して、すべてのアプリケーションの一覧を管理することをお勧めします。
- ビジネスケイパビリティとアプリケーションをリストアップする際に、それらのアプリケーションが1つのビジネスケイパビリティだけに割り当てられているのか、複数のビジネスケイパビリティ間で共有されているのか、それともサービスモデル内で使用されているのかを確認してください。どのアプリケーションがビジネスケイパビリティ間で共有されているのか、どのビジネスケイパビリティがバリューストリーム間で共有されているのかを明確にします。チーム間で共有される大規模なアプリケーションについては、データをそれぞれ別のオーナーシップを持つ、自己完結型のコンテキスト境界に分割します。これについては、8章で後ほど補足します。
- データ消費を図式化し、データ消費とデータ作成がどのように整合するかをマッピングします。境界線は、データ処理やデータの調和など、消費側の要求によって定義することもできます。この作業では、共有または重複するデータに関する懸念事項を洗い出すことが重要です。適合性と自律性についてはこの後の段階でバランスを取ります。これについては10章で説明します。
- （データ）ドメインを定義してください。経験則では、それぞれのドメインはビジネスケイパビリティの実装と整合させてください。この解決空間では、複数のアプリケーションとコンポーネントが連携して特定のビジネス価値を提供します。この作業では、コンテキスト境界、ビジネスケイパビリティ、チームの整合性をとるように心がけましょう。ここで学んだように、コンテキスト境界とは、ビジネスケイパビリティをカプセル化したものです。それぞれのビジネスケイパビリティは少なくとも1回は実装（インスタンス化）されます。チームにドメインの視点を浸透させるために、コンテキスト境界は1つのチームが所有することを原則とします。ただし、1つのチームが複数のコンテキスト境界を所有してもかまいません。
- ドメインは重なるべきではありません。このような問題が発生した場合、組織構造をドメイン構造に合わせるか、または調整しなおす必要があります。中央集権的な運用チームや開発チームからなる伝統的な企業構造では、機能横断的なチームからなるフラットな組織構造に比べると、ビジネスドメインに合わせたモデル化が難しいでしょう。バーチャルチームでの

連携を選択するか、チーム間で人を移動させるか、目標状態に向けた段階的な移行をどのように行うかを検討してください。

- コンテキスト境界とは、ポリシーと標準が適用される領域のことです。管理を行う際には、ケイパビリティマップを使ってビジネスケイパビリティを図示し、これをポリシーと標準を適用する際の入力として使います。ドメインは、ビジネスケイパビリティレベルで設定したポリシーと標準を継承します。したがって、このマップは、どの領域にどのような標準が適用されるかを示したガイドとなります。
- 境界を越えたときに何が起こるかを明文化してください。つまり、ドメインがどのような標準や原則に従わなければならないかを明確にします。ドメイン境界を越えてデータを流通し、アプリケーションを統合することは、さまざまな理由により難しくなります。例えば、ドメイン内で運用するために設計されたシステムは、簡単には相互運用できないかもしれません。プロセスを連動させるのが難しい場合もあるでしょう。また、単に関係者が遠く離れていて、お互いをよく知らないということもあるかもしれません。これはリスクのある部分であり、さまざまな障害を乗り越える必要があります。

現状を正確に包括的に把握することこそ、デジタルトランスフォーメーションの土台となります。現在の状況をはっきりと理解せずに、将来の最適化された状態を目指そうとしてもそれは無理な話です。現状が把握できれば、組織としてどんな変化を起こせるのか、その計画を練ることができます。

2.6　まとめ

本章で学んだことを振り返ってみましょう。新しい設計をうまく実行するには、すべてを、そして間違いなくすべてを、ビジネスケイパビリティを中心に構築しなければならないということです。**ビジネスに焦点を当てた**アプローチこそが、本当の価値の提供へとつながるのです。したがって、ビジネスケイパビリティを重要なビジネス戦略に紐付けなければなりません。ビジネスケイパビリティそのものが問題領域を定義します。ビジネスケイパビリティはアーキテクチャを定義するドメインです。システム、データベース、アプリケーション、プロセス、API、チーム、データプロダクトはすべて、これらのドメインにマッピングされます。マイクロサービスの中でよく採用されるDDDは、複雑さを洗い出し、アプリケーションのインタフェースを紐解くのに役立ちます。

その次に、強固な境界を設定することについて学びました。連合型モデルにおいて、ドメインやビジネスチームがそれぞれ個々にデータを変更したり交換したりできるようにすることで、柔軟性が増し、ドメインが並行して作業できるようになります。連合型モデルのドメインはそれぞれ、アーキテクチャの一部を所有し管理します。アプリケーションをビジネスケイパビリティのレベルで論理的にグループ化することで、同じ関心事を共有するアプリケーションはまとめて管理され、ビジネス関心事が異なるアプリケーションとの直接的な依存関係はなくなります。この手法により、チームは単一のビジネスケイパビリティの価値を提供することに完全に集中できます。そのため、組織の目的が明確になり、アジリティがもたらされます。各ドメインは、コンテキスト境界によって論理的にグループ化され、アーキテクチャ全体のある特定の部分を管理します。この管理対象には、権威ある一意のデータセットも含まれます。最後に、ドメインと分離に関するこの議論は、データメッシュアーキテクチャに限ったものではない、ということは覚えておいてください。明確に境界線を設定することは、データ管理やアプリケーション統合に関するどのようなパラダイムにおいても必要なことです。

ドメインアーキテクチャを完成させるのは簡単なことではありません。まず、エンタープライズアーキテクチャという分野からスタートします。エンタープライズアーキテクチャのリーダーは、変革プロセスを推進し、ビジネスビジョンを設計に確実に落とし込む必要があります。また、標準を中央で定めることにも着手します。データ駆動型に移行するには、それぞれのチームが、境界の引き方や、データの所有、管理、流通、消費方法について決定する権限を放棄する必要があります。オーナーシップを確立するには、規律を守り、日々実践する必要があります。このような責任分担方法に移行する際には抵抗が予想され、組織として政治的な決断を迫られるかもしれません。強力なガイダンスがなければ、組織は変革できず、アーキテクチャはその能力を十分に発揮できません。境界を無視し、データサービスを自ら配備し、ポイントツーポイントの統合を行うチームは、「データスプロール」のリスクを抱えることになります[†12]。再利用性や、一貫性、データ品質は、新しいアーキテクチャにおいて重要な観点です。スケールアップのためには、絶え間ないコミュニケーション、貢献、そして強力なガバナンスが必要です。

[†12] **データスプロール**とは、企業が日々生み出している膨大なデータ量とデータの種類のことを指します。

さらに、このようなデータランドスケープの近代化には、現実的なアプローチが必要です。なぜなら、密結合されたランドスケープからの移行は非常に難しいからです。段階的に始めて、徐々に規模を拡大していくことで、ドメインやユーザーにメリットを認識してもらい、組織の競争力を高める新しいアーキテクチャに貢献したいと考えてもらうようにします。そして次第に、新しいアーキテクチャを拡張し、ドメインを増やし、統合パターンを追加し、機能を強化します。このステップバイステップの手法については、12章で理論を実践に移す際に再び取り上げます。

この章では、ドメイン駆動設計、ビジネスアーキテクチャ、データ共有、クロスドメインコラボレーションモデルなど、たくさんの理論を紹介しました。少し概念的すぎると感じられた方もご安心ください。次の章では、再びヘリコプターに乗り込み、解決空間を見渡し、ランディングゾーンや、データプロダクト、API、イベントについて説明します。一段低いところを飛ぶことになりますが、それでも雲の上にいるため、見晴らしは良いでしょう。